

Patrones de diseño comunes

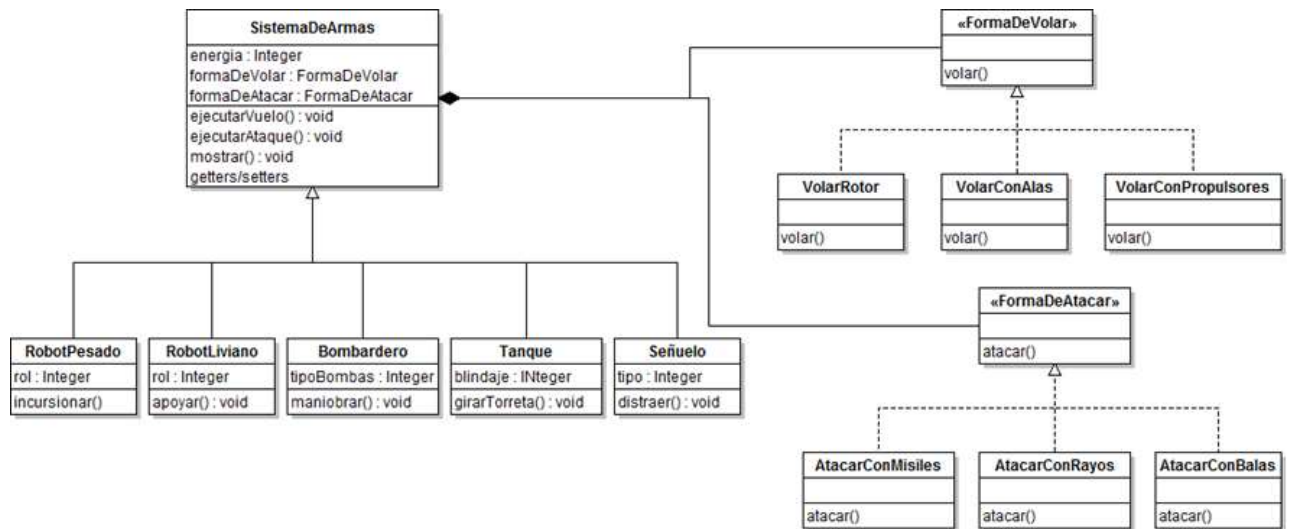
Índice

Strategy.....	2
Patrón Strategy: enunciado	3
UML estándar del Patrón Strategy	3
Template Method	3
Qué se logra aplicando Template Method	6
Template Method: enunciado	6
UML estándar del patrón Template Method	7
Patrón State	7
Patrón State: enunciado	10
Patrón State: conclusiones	10
Relaciones con otros patrones	10
Patrón Singleton	11
¿Por qué necesitamos una sola instancia de un objeto?	11
Acceso global a una variable en Java: algunas consideraciones	11
Creando un objeto sobre demanda (Lazy Loading).....	12
Patrón Singleton: enunciado	12
Patrón Singleton: conclusiones	12
¿Qué se puede hacer?	13

Strategy

Anteriormente enumeramos algunos principios de diseño. A su vez, introdujimos las ventajas de favorecer la composición por sobre herencia. Esto, sumado al principio de “programar contra interfaces”, hizo que arribáramos a un diseño, en el que separamos comportamientos en “familias” y nos permita alternar estos comportamientos en tiempo de ejecución.

Si vemos todo el panorama completo, veríamos algo así:



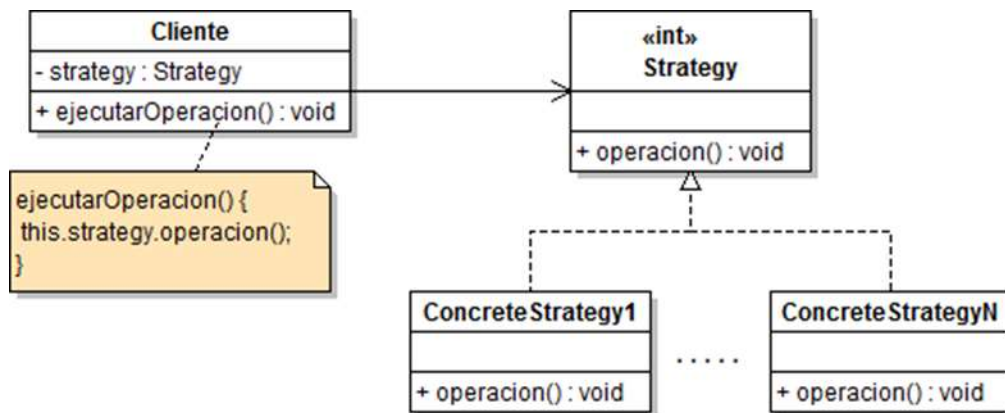
Si generalizamos el problema, podemos pensar en las diferentes “formas de” como diferentes “algoritmos” o “familias de algoritmos”. Es decir, cada “algoritmo” representa algo que un sistema de armas puede hacer.

Componer nos da mayor flexibilidad. Heredar nos “encerraba” en comportamientos fijos. Componer nos permite encapsular los comportamientos, permitiendo intercambiarlos sin afectar otras partes del sistema y, más aún, nos permite cambiar los comportamientos en RUNTIME (siempre que se respeten las interfaces). Esto da lugar a otro PRINCIPIO DE DISEÑO: favorecer la composición por sobre la herencia.

Patrón Strategy: enunciado

Con todo lo visto anteriormente, y llegando a esta conclusión, enunciar el patrón **STRATEGY**: **Strategy define una familia de algoritmos encapsulados de forma tal que pueda intercambiarse. El patrón Strategy permite variar entre diferentes algoritmos sin afectar a los clientes que los usan.**

UML estándar del Patrón Strategy



Template Method

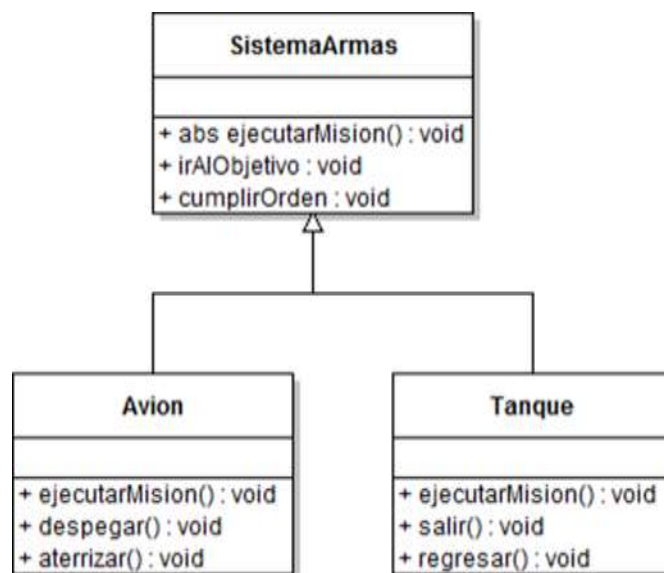
Habiendo encapsulado algoritmos con el Strategy, podemos pensar en encapsular las diferentes partes de un algoritmo complejo. De esta manera, podemos proveer “hooks” (o “enganches”) sobre las partes del algoritmo o los pasos del mismo, como para reemplazarlas o extenderlas sin alterar el algoritmo original.

Volviendo a los vehículos armados de nuestros ejemplos, podemos ver que cada vehículo puede ejecutar una misión de formas variadas, pero similares entre sí. Por ejemplo, un avión debería salir de la base (típicamente despegar...), ir hasta el objetivo, cumplir la orden, regresar (y aterrizar...). Un tanque debería salir de la base, ir hasta el objetivo, cumplir la orden y regresar. Un barco..., un robot... son todos bastante similares.

Veamos, por ejemplo, qué pasaría si tenemos que codear un avión y un tanque para nuestro juego de guerra en tiempo real:

Avión	Tanque
<pre> public class Avion { ejecutarMision() { despegar(); irAlObjetivo(); cumplirOrden(); aterrizar(); } despegar() { syso("Salgo del hangar"); syso("despego"); } irAlObjetivo() { syso("voy al objetivo"); } cumplirOrden() { syso("cumplo orden"); } aterrizar() { syso("vuelvo volando"); syso("aterrizo"); } } </pre>	<pre> public class Tanque { ejecutarMision() { salir(); irAlObjetivo(); cumplirOrden(); regresar(); } salir() { syso("Salgo de la base"); } irAlObjetivo() { syso("voy al objetivo"); } cumplirOrden() { syso("cumplo orden"); } regresar() { syso("me desplazo a la base"); } } </pre>

Viéndolo así, lado a lado, es evidente que se duplica código. ¿Cómo podríamos solucionar el problema entonces?

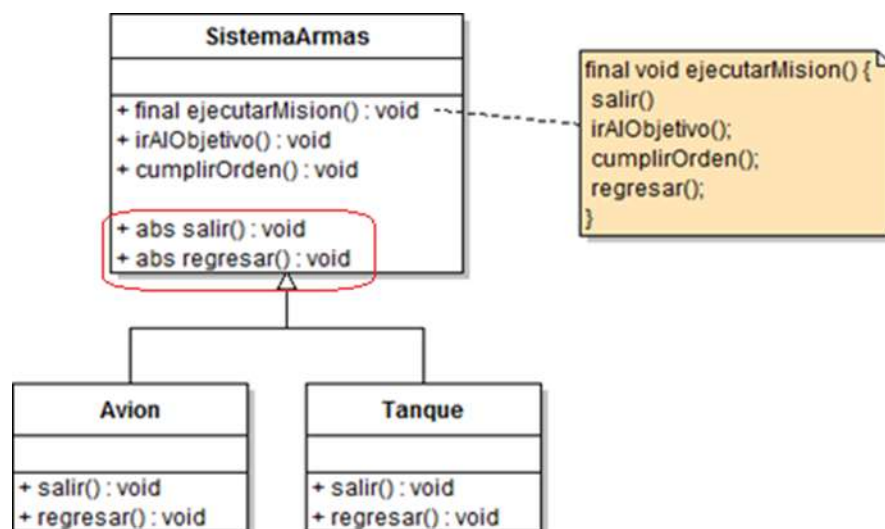


En este caso, el comportamiento `ejecutarMision()` es abstracto, porque cada subclase lo llevará a cabo distinto, por lo que debe implementarse. En cambio, `despegar()`, `aterrizar()`, `salir()` y `regresar()` son específicos de cada uno, por lo que están definidos y codeados en las subclases. A simple vista, podemos notar que se relacionan los conceptos “salir” con “despegar”, y “aterrizar” con “regresar”. Es decir, el MECANISMO (algoritmo) de ejecutar una misión siempre tiene los mismos pasos: comenzar la misión [`comenzarMision()`], ir al objetivo [`irAlObjetivo()`], cumplir la orden [`cumplirOrden()`] y volver a la base [`volverABase()`]. Entonces podemos abstraer el `ejecutarMision` sin tener que andar redefiniéndolo en cada hija.

Entonces, dejemos que el algoritmo siempre ejecute los mismos pasos:

```
ejecutarMision() {
    comenzarMision();
    irAlObjetivo();
    cumplirOrden();
    volverABase();
}
```

Y relegar a clases hijas la decisión de cómo llevar a cabo ciertos pasos. Por ejemplo, digamos que el salir y el regresar son solo las partes (pasos) que cambian del algoritmo de ejecutar una misión (un avión despegue, un tanque no / un avión aterriza, un tanque no).



El método “ejecutarMision” es lo que se conoce como Template Method. El Template Method presenta, como su nombre lo indica, una plantilla. El TM presenta una plantilla para un algoritmo, indicando sus pasos. Uno o más de esos pasos pueden ser redefinidos por subclases. Para hacer cumplir esto se marcan como abstractos.

Redefinir todos los pasos sería como reemplazar por completo el método. Si se permitiera redefinir el todo el método plantilla, estaríamos en presencia de un Strategy. Para evitar tal situación es que se marca final.

Qué se logra aplicando Template Method

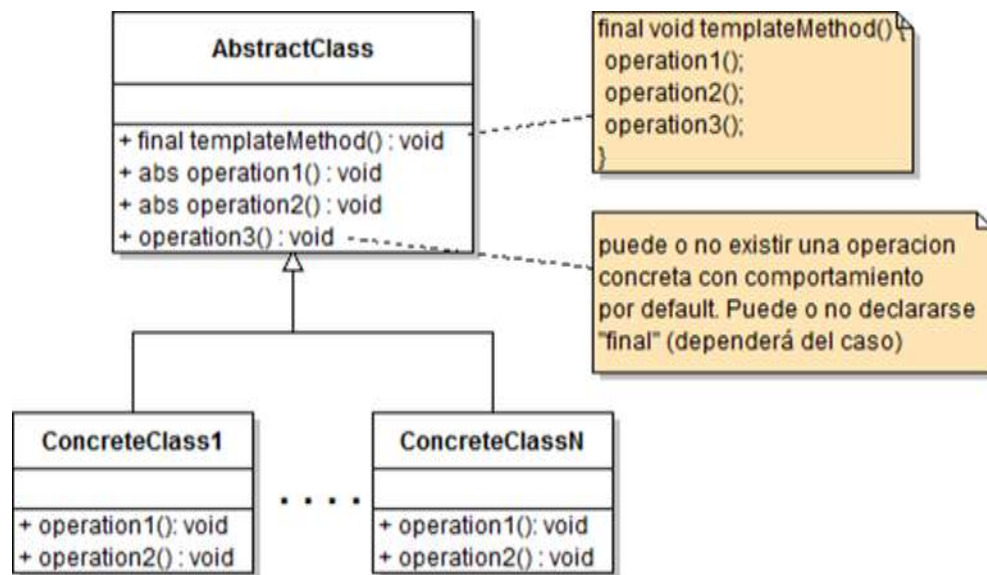
Quedó encapsulado el algoritmo, mejorando la reusabilidad de este. Cualquier cambio se hace en un solo lugar. La clase con el TM concentra el conocimiento del algoritmo, dejando a las subclases la implementación concreta de este.

Ahora, el TM brinda una plantilla, un framework, que cualquier “sistemaArmas” puede tomar para funcionar, implementando algunos (no todos) de los métodos correspondientes del algoritmo.

Template Method: enunciado

Template Method define el esqueleto, la plantilla de un algoritmo en un solo método cediendo a las subclases las implementaciones de algunos pasos. Es decir, se permite a las subclases redefinir algunos pasos de un algoritmo sin alterar la estructura de este.

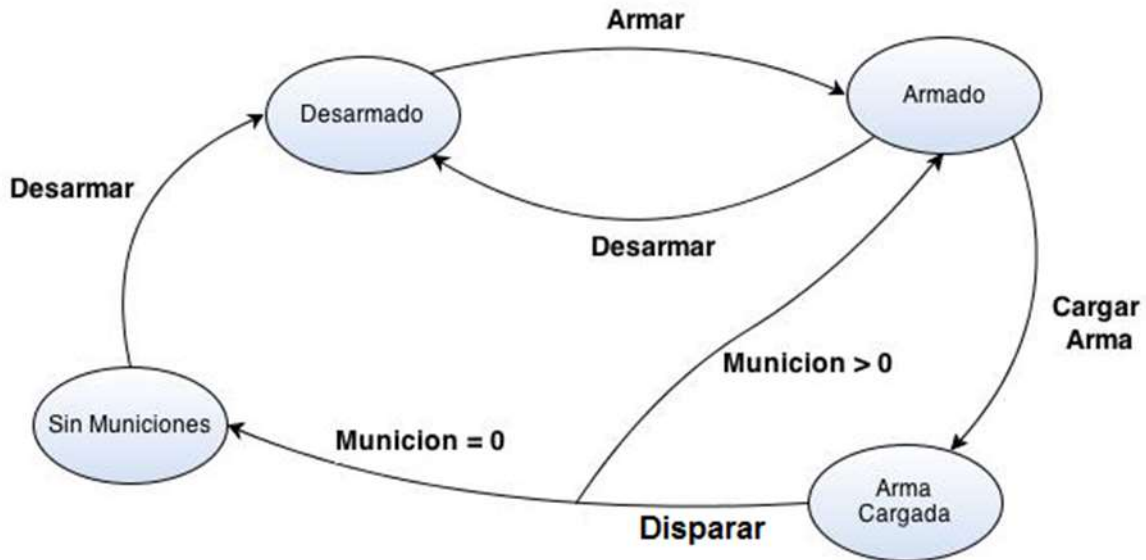
UML estándar del patrón Template Method



Patrón State

Imagínense uno de los sistemas de armas discutido. Supongamos un “tanque”. Simplificando la automatización del sistema de tiro podemos decir: se carga el arma, se dispara y vuelve a empezar. Ahora, un sistema así de simple puede pasar por diferentes situaciones: puede estar “armado” (listo para entrar en combate) o “desarmado” (en modo de espera o apagado), puede tener munición para disparar o no, puede tener la munición lista o puede estar sin municiones, etc., etc. A su vez, cada vez que alguien interactúa con el sistema de tiro, este cambia. Por ejemplo, si le quedara una sola bala, al dispararla queda el tanque vacío o “sin munición”.

Esta situación, suena a estados y transiciones de estados: el tanque pasa por diferentes estados, cada vez que “hace algo”.



A simple vista se ve en qué “estado” puede estar la máquina y las operaciones que puede realizar. Entonces, si se modelara el sistema de armas como un objeto, este tendría estado interno y operaciones que invocarle para cambiar ese estado.

El *approach* más sencillo sería tener un atributo “estado” y después un método para cada transición; a su vez, dentro de ese método, un IF-ELSE para preguntar “si está en el estado A, B, C, hago una cosa u otra, pero solo si está en el estado X, paso al Y”. Lo que no tiene nada de malo, estamos almacenando el estado interno y escribiendo código condicional que actúe según los diferentes estados. El problema aparece cuando agregamos un estado o quitamos uno... hay que cambiar el if-else (o lo que es peor, puede afectar toda la lógica...). Sería interesante que el sistema se comporte distinto según el estado en el que estoy y que dependa el estado en el que se encuentra en un determinado momento, ya no de una lógica centralizada. Es decir, quiero desacoplar el estado actual del comportamiento del sistema.

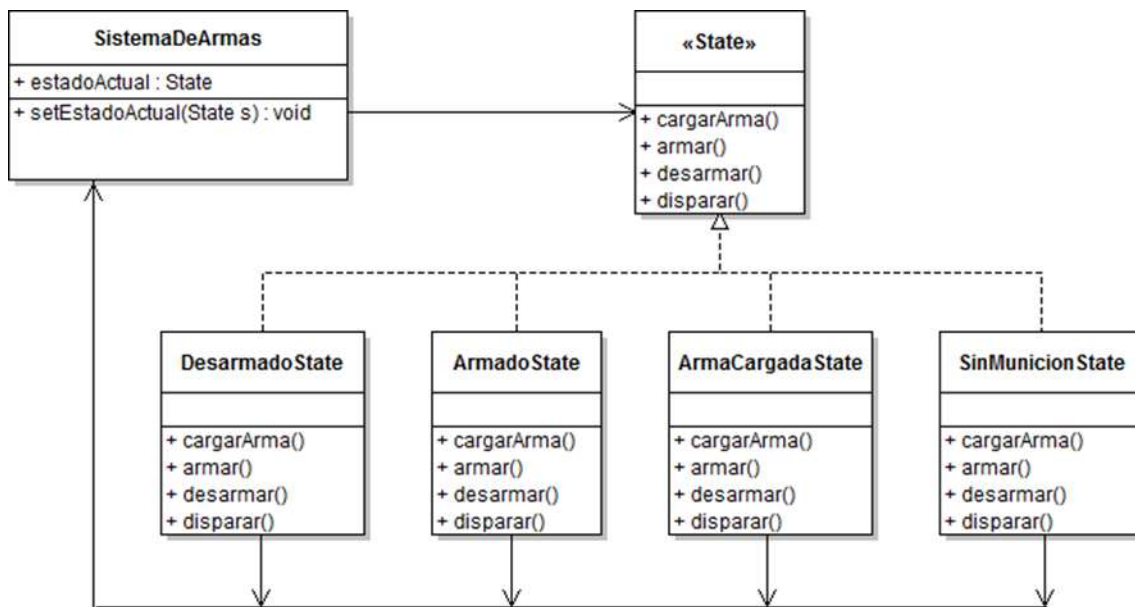
Así que lo que podemos hacer es encapsular los posibles estados en sus propias clases y posiblemente encapsular también a quien se encargue de cambiar los estados.

Para ello:

1. Definamos una interfaz que contenga cada transición de estado, es decir, cada operación que implique un cambio de estado.

2. Implementar cada uno de los estados posibles en una clase. Cada una tendrá la responsabilidad de actuar según el estado en que se encuentre la maquina (el problema).
3. Eliminar todo tipo de código condicional.

Entonces, cada estado, y cómo actuará la maquina en cada estado, estará concentrado en clases y no dispersado a través de varias sentencias IF-ELSE.



El comportamiento del sistema de armas depende del estado en el que está. Ese comportamiento puede incluir una transición a otro estado. En algunos casos puede ser mejor que los cambios de estado los administre el propio sistema.

Poner la lógica de transiciones de estado en cada uno de los “xyzState” aporta mayor flexibilidad, dado que cada uno conoce por qué cambia, cuándo lo hace y hacia qué otro estado va. Por otro lado, se acopla un poco, al tener que hacer que un “xyzState” conozca al menos a otro “xyzState”.

Patrón State: enunciado

El patrón STATE permite a un objeto alterar su comportamiento de acuerdo con su estado interno.

Patrón State: conclusiones

Da una clara visión de qué hace la máquina en cada estado. Es decir, se ve claro en qué situación se encuentra el contexto/problema → normalmente, se usarían constantes + IF/ELSE, pero la lógica está distribuida a lo largo de varios métodos, acoplando todo por todos lados y es susceptible a errores, más aún: complica a futuro si se quieren agregar o sacar estados.

Al estar los estados encapsulados se reduce la posibilidad de errores en la codificación y dejando al contexto en un estado “inconsistente”.

Da una clara visión de los cambios de estado. Usando constantes para los estados junto con IF/ELSE, el paso de estado se puede confundir con una asignación de valor a una variable.

Sin embargo, no todo es perfecto. Hay una desventaja innegable: a medida que crece la cantidad de estados, se incrementa la cantidad de clases, por lo q se debe escribir más código y a su vez resulta en más objetos, consumiendo más memoria.

Relaciones con otros patrones

Con Strategy: son iguales en forma, pero se diferencian en intención.

Con el State, el estado está encapsulado en objetos. Con los cambios de estado, el contexto varía su comportamiento. El encapsulamiento mencionado hace que el cliente no conozca casi nada acerca de los objetos xyzState

Por otro lado, en el Strategy, el cliente es quien especifica el objeto xyzStrategy que se va a utilizar en el contexto. Si bien se puede alterar la estrategia en *runtime*, normalmente hay una estrategia adecuada para cada problema.

Esta bueno pensar al Strategy como una herencia. Una vez elegida la estrategia, queda fija hasta que termina el problema. Para cambiar la estrategia hay que componer el contexto con un objeto diferente.

El State sirve como para reemplazar varios condicionales que alteran el funcionamiento del contexto. Al cambiar el contexto de estado, cambia la forma de comportarse.

Patrón Singleton

¿Por qué necesitamos una sola instancia de un objeto?

Puede ser objetos que sean contenedores de preferencias, pueden ser diálogos, editores, mensajes, etc., objetos que usen recursos limitados, historiales de acciones o cuya construcción sea costosa y no varíe durante el ciclo de vida de la aplicación. Por otro lado, y en este último caso, puede que no necesitemos ese objeto hasta cierto punto en la aplicación y crearlo apenas arranca podría significar un tiempo de espera inaceptable para el usuario.

En cualquiera de estos escenarios, sería útil poder tener acceso global a ese objeto, ya que contamos con una única instancia.

Acceso global a una variable en Java: algunas consideraciones

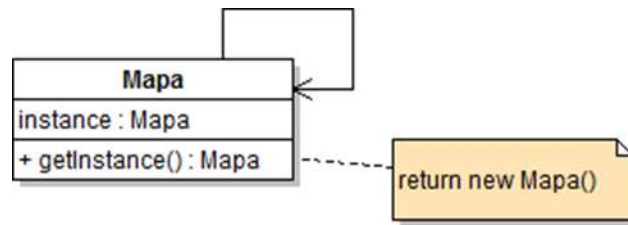
¿Qué pasaría si aquello guardado en la variable global fuera muy costoso en recursos? Quedaría “atado” a la variable hasta que el programa no termine y puede ser que solo necesite que quede accesible después de un tiempo de usar la aplicación (“lazy initialization”). Usando variables estáticas, no resultaría fácil controlar cuando o a partir de cuándo se inicializa.

Si uso variables public static, rompo con cualquier concepto de encapsulamiento y ocultamiento de información, dado que esos valores pueden ser accedidos por cualquier objeto sin mayor control.

Creando un objeto sobre demanda (Lazy Loading)

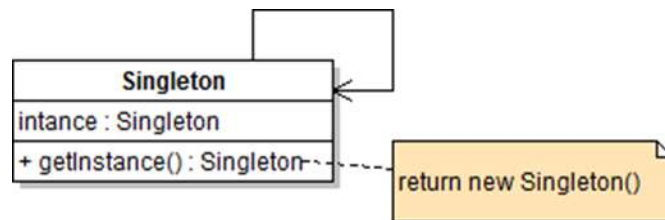
En nuestro juego, que se trata de un mundo futuro en guerra, podemos mostrar “mapa” con la situación de los bandos, los ejércitos, etc. Pero necesito solo un mapa y mantener en este todo actualizado al momento de seleccionar la opción “mostrar mapa” de la pantalla.

Entonces, puedo implementar el patrón Singleton:



Patrón Singleton: enunciado

Singleton provee la certeza de que haya siempre SOLO UNA INSTANCIA de una clase determinada y, a la vez, provee un punto de acceso global a ella.



Patrón Singleton: conclusiones

Solo la clase Singleton crea el Singleton. Esto se debe a que el patrón se implementa incluyendo un constructor privado. Dado que nadie puede llamar al constructor directamente, lo único que hacen los clientes del Singleton es obtener una referencia a éste mediante el método provisto getInstance()

Ahora, el acceso es global como si fuera una variable, con la diferencia de que se trata de una clase como cualquier otra con todos los beneficios: atributos, getters y setters para cada uno y métodos.

Un potencial problema es la creación del objeto único. Si este necesita ser accedido por varios *threads* a la vez, ya sea para escribir o leer o incluso al momento de obtener la referencia puede haber problemas. Hay que poner especial cuidado en estos casos y hacer uso de las herramientas que da el lenguaje para prevenirlos. Típicamente, agregaríamos `synchronized` (que evita este tipo de situaciones problemáticas. Ver módulo 11, acerca de Hilos de Ejecución) al `getInstance()` y listo. Pero solo sería relevante la primera vez que se corra, que es cuando se instancia la clase y se asigna a la variable. A partir de ahí, sincronizar las llamadas es inútil y costoso a la vez (porque hace de esta llamada un cuello de botella).

¿Qué se puede hacer?

1. Si la performance no es un tema importante, dejarlo con el `synchronized` en el `getInstance`. Tener en cuenta que sincronizar un método lo vuelve más lento.
2. Eliminar el chequeo por null, haciendo que se no se instancie la clase sobre demanda, sino dejarla instanciada. O sea, `private static Singleton = new Singleton()` (a esto se le llama “eager initialization” en contraposición con el “lazy loading”)
3. Usar “double-checked locking” para reducir la sincronización. Con esta técnica primero se verifica que si la instancia esta creada, y si no, solo entonces se realiza la sincronización. El código quedaría:

```
public class Singleton {  
  
    private volatile static Singleton uniqueInstance;  
  
    private Singleton() {} //constructor PRIVADO!  
  
    public static Singleton getInstance() {  
        if (uniqueInstance == null) {  
            synchronized (Singleton.class) {  
                if (uniqueInstance == null) {  
                    uniqueInstance = new Singleton();  
                }  
            }  
        }  
        return uniqueInstance;  
    }  
}
```

En este código se pueden ver cuatro temas importantes:

- El uso de volatile: volatile se asegura de que multiples threads manejen la variable instance de forma correcta cuando se le esté asignando la instancia de Singleton.
- Luego, el getInstance se hace como siempre, pero solo luego del chequeo por null se sincroniza el bloque de código.
- El bloque de instanciación chequea por segunda vez si la variable es null. Solo entonces hace el new Singleton().
- Después del chequeo por partida doble, se retorna la instancia.

De esta manera nos aseguramos de que el acceso por múltiples threads sea seguro, a la vez que mejoramos la performance de la opción 1.

Si se trata de armar Singleton en contextos de múltiples threads, con estas técnicas puede solucionarse el problema.

Otro potencial problema es que el Singleton rompe con el principio de una clase<->una responsabilidad (es decir, se acopla). La clase Singleton hace lo que tiene que hacer además de gestionarse como instancia única. El caso de Singleton puede ser una excepción. A veces es mucho más simple usar Singleton que otra solución y toma precedencia por sobre el principio.

Un problema adicional se presenta si queremos reutilizar la funcionalidad del Singleton mediante la herencia. Es imposible, dado que el constructor es privado. De hacerlo público rompemos con el propósito del Singleton. No solo por hacer el constructor visible, sino porque las hijas compartirán la variable y probablemente no sea lo que queramos. Habría que implementar algún tipo de contenedor y eso complica las cosas cuando, en realidad, decidimos usar Singleton para simplificarlas.